Assignment-5

Parallel & Distributed Computing
(UCS645)

Submitted by:
Bhawesh War - 102313065

Submitted to:
Dr. Saif Nalband

Performance Analysis

1. DAXPY Operation

Processes	Time (s)	Speedup	Efficiency
1	5.7766e-05	1.00	1.00
2	5.4683e-05	1.06	0.53
4	3.57499e-04	0.16	0.04
8	3.6566e-04	0.15	0.02

Observations:

- Slight improvement from 1 to 2 processes.
- Performance degrades beyond 2 processes.
- Communication overhead dominates computation.
- Small workload limits parallel efficiency.
- Efficiency decreases sharply with increasing processes.

2. Broadcast Communication

Processes	Manual Time (s)	MPI_Bcast Time (s)
2	0.0147691	0.0163098
4	0.0903788	0.0793957
8	0.380115	0.112594

Observations:

- At 2 processes, manual broadcast is slightly faster due to minimal communication overhead.
- At 4 processes, MPI_Bcast starts outperforming manual broadcast.
- At 8 processes, MPI_Bcast is significantly faster than manual broadcast.
- Manual broadcast time increases rapidly with number of processes (linear growth).
- MPI_Bcast scales better due to its tree-based communication algorithm (logarithmic growth).
- The performance gap widens as the number of processes increases.
- MPI_Bcast is more efficient and scalable for larger parallel systems.

Additional Questions' Answers :

- **Graph (Execution Time vs Number of Processes):**

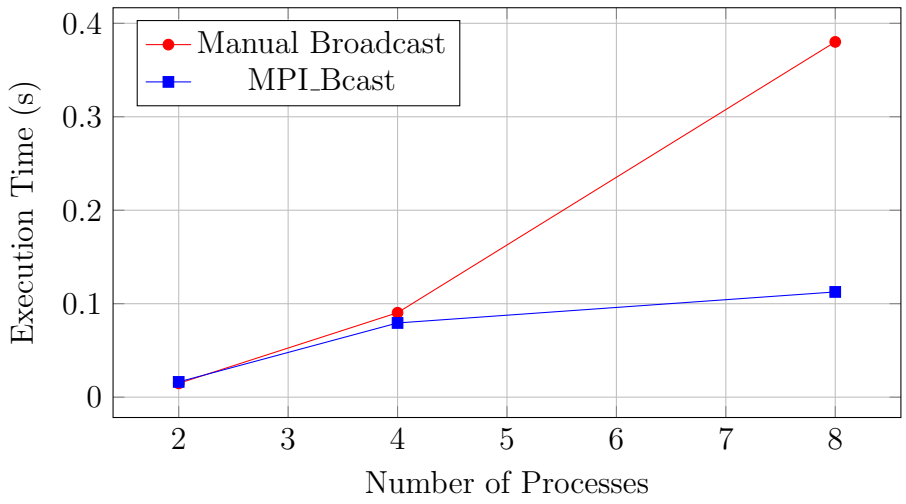

Figure 1: Execution Time Comparison

- The for-loop broadcast scales linearly due to sequential sends from Rank 0, giving O(p) complexity.
- With an increase in number of processes, MPI_Bcast scales better due to a tree-based approach with O(log p) complexity. In contrast, the manual for-loop broadcast sends data from the root process to each process one by one, resulting in linear growth with O(p) complexity.

3. Distributed Dot Product

Processes	Time (s)	Speedup	Efficiency
1	0.822628	1.00	1.00
2	0.413755	1.99	0.99
4	0.507905	1.62	0.40
8	0.444575	1.85	0.23

Observations:

- Near ideal speedup at 2 processes.
- Performance slightly degrades at higher processes.
- Large workload benefits from parallelization.
- Communication overhead reduces efficiency.
- Efficiency decreases as processes increase.

Additional Questions' Answers :

- **Graph (Speedup vs Number of Processes):**

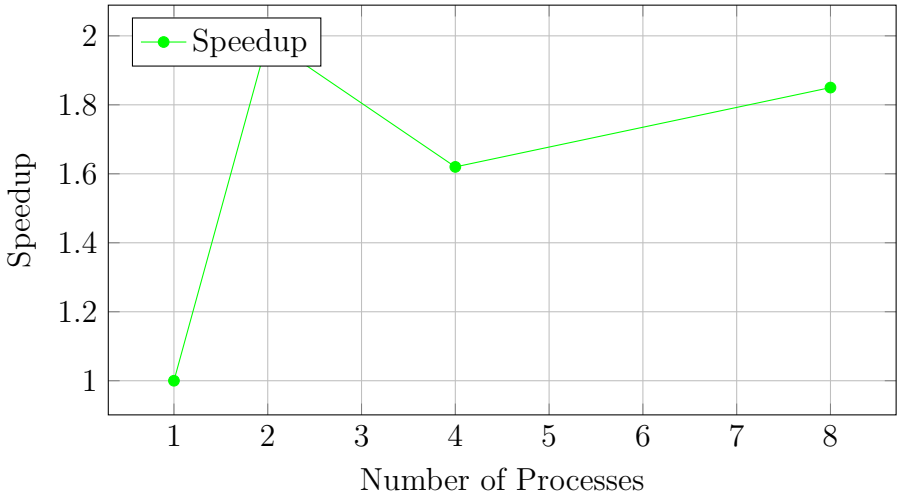

Figure 2: Speedup vs Processes

- Speedup values:
  - For 2 processes: $S \approx 1.99$
  - For 4 processes: $S \approx 1.62$
  - For 8 processes: $S \approx 1.85$
- Parallel Efficiency:
  - For 2 processes: $E \approx 0.99$
  - For 4 processes: $E \approx 0.40$
  - For 8 processes: $E \approx 0.23$
- Perfect linear speedup was not achieved.
  - According to Amdahl's Law, the serial portion limits scalability.
  - Communication overhead (MPI_Bcast, MPI_Reduce) and synchronization delays reduce performance.
  - Network latency and process management overhead further decrease efficiency.

4. Prime Number Computation

Processes	Time (s)	Speedup	Efficiency
2	0.0310501	1.00	0.50
4	0.130134	0.24	0.06
8	0.149134	0.21	0.02

Observations:

- Best performance at 2 processes.

- Significant slowdown at higher processes.
- Master becomes a bottleneck.
- Communication overhead dominates.
- Load imbalance affects performance.

5. Perfect Number Computation

Processes	Time (s)	Speedup	Efficiency
2	0.0959073	1.00	0.50
4	0.149133	0.64	0.16
8	0.0779035	1.23	0.15

Observations:

- Performance decreases from 2 to 4 processes.
- Slight improvement at 8 processes.
- Irregular behavior due to dynamic workload.
- Master-worker overhead impacts efficiency.
- Limited scalability for small input size.

Conclusion

- Parallel performance improves only for large computational workloads.
- Communication overhead significantly impacts performance.
- Speedup is not always proportional to number of processes.
- Efficiency decreases with increasing processes.
- MPI_Bcast demonstrates better scalability than manual communication.
- Master-worker model introduces bottlenecks at higher process counts.